

Convolutional Neural Network 기초

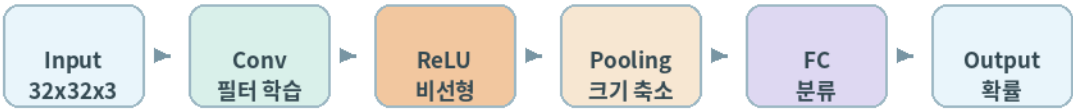
이미지 구조, 합성곱, 필터, Feature Map, Pooling, CNN 학습 과정

이 문서는 DNN의 기본 개념을 이해한 뒤, 이미지 데이터에 특화된 CNN이 왜 필요한지 설명한다. 단순히 CNN 용어를 외우는 것이 아니라, 필터가 이미지를 어떻게 훑고 Feature Map이 어떻게 만들어지는지를 그림 중심으로 다룬다.

학습 목표

CNN이 DNN과 다른 이유를 설명할 수 있다. Convolution, Filter, Feature Map, Stride, Padding, Pooling의 역할을 이해할 수 있다. 이미지 분류에서 CNN이 공간 구조를 어떻게 활용하는지 말할 수 있다.

CNN 전체 구조



이미지의 공간 구조를 유지하면서 지역 패턴을 단계적으로 추상화한다.

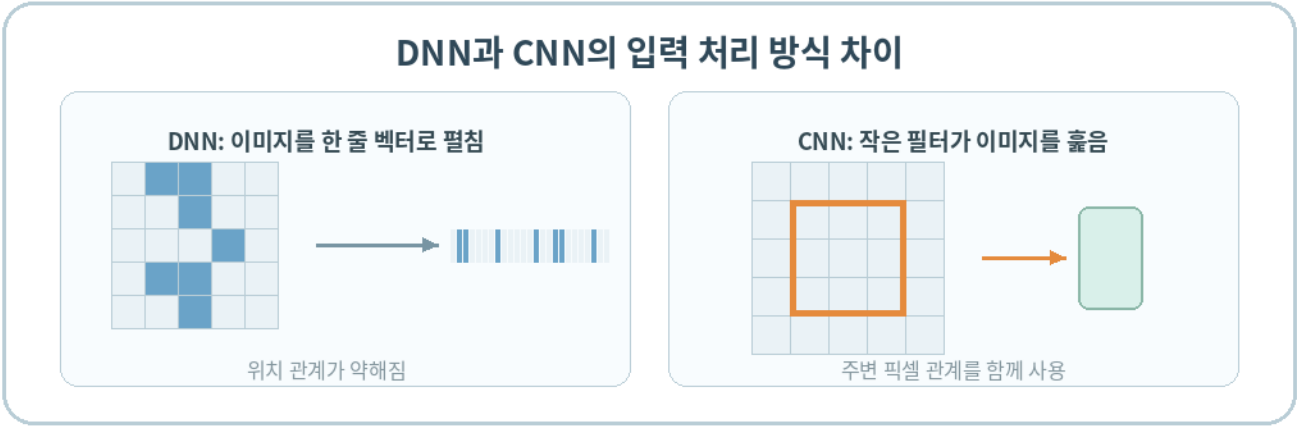
1. 왜 CNN이 필요한가?

DNN도 이미지를 분류할 수 있다. 하지만 일반 DNN은 이미지를 한 줄 벡터로 펼쳐서 입력하기 때문에, 픽셀의 위치 관계와 주변 픽셀의 패턴을 직접 활용하기 어렵다.

CNN은 이미지를 펼치지 않고 2차원 공간 구조를 유지한 채 작은 필터로 지역 패턴을 탐지한다. 그래서 이미지의 모서리, 선, 질감, 부분 형태 같은 특징을 더 효율적으로 학습할 수 있다.

핵심 정의

CNN = 이미지의 공간 구조를 유지하면서 작은 필터를 이동시켜 지역 특징을 추출하는 신경망 구조



2. 이미지 데이터의 구조: $H \times W \times C$

이미지는 사람에게서는 그림처럼 보이지만, 컴퓨터에게는 숫자 배열이다. 흑백 이미지는 각 위치의 밝기값으로 이루어진 2차원 배열이고, 컬러 이미지는 R, G, B 세 채널이 쌓인 3차원 배열이다.

CNN은 이 구조를 유지한 채 계산하기 때문에 이미지의 위치 정보와 주변 관계를 사용할 수 있다.

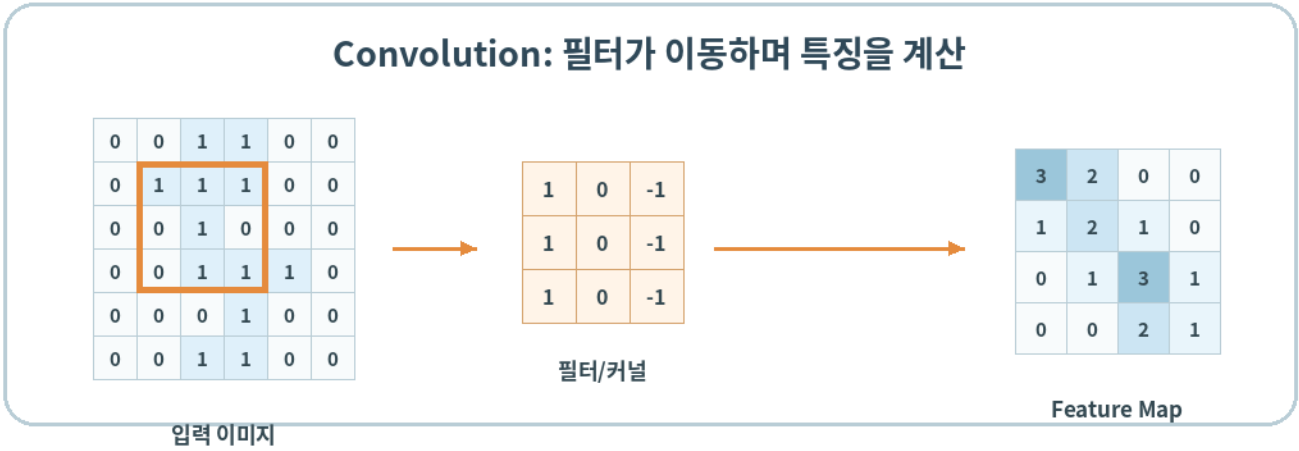


표현	의미	예시
H	이미지의 높이	28 또는 32
W	이미지의 너비	28 또는 32
C	채널 수	흑백 1, 컬러 3

3. Convolution이란 무엇인가?

Convolution은 작은 필터를 이미지 위에서 이동시키며, 필터가 보는 작은 영역과 필터 값을 곱해 더하는 계산이다. 이 계산 결과가 한 위치의 특징 반응값이 된다.

필터가 이미지 전체를 훑으면 Feature Map이 만들어진다. 필터는 고정된 규칙이 아니라 학습을 통해 바뀌는 가중치이다.



$$y[i,j] = \sum_u \sum_v x[i+u, j+v] \cdot k[u,v]$$

4. 필터는 작은 패턴 탐지기이다

필터는 이미지의 특정 패턴에 강하게 반응하는 작은 가중치 묶음이다. 어떤 필터는 세로선을 잘 찾고, 어떤 필터는 가로선이나 모서리, 색 대비에 반응할 수 있다.

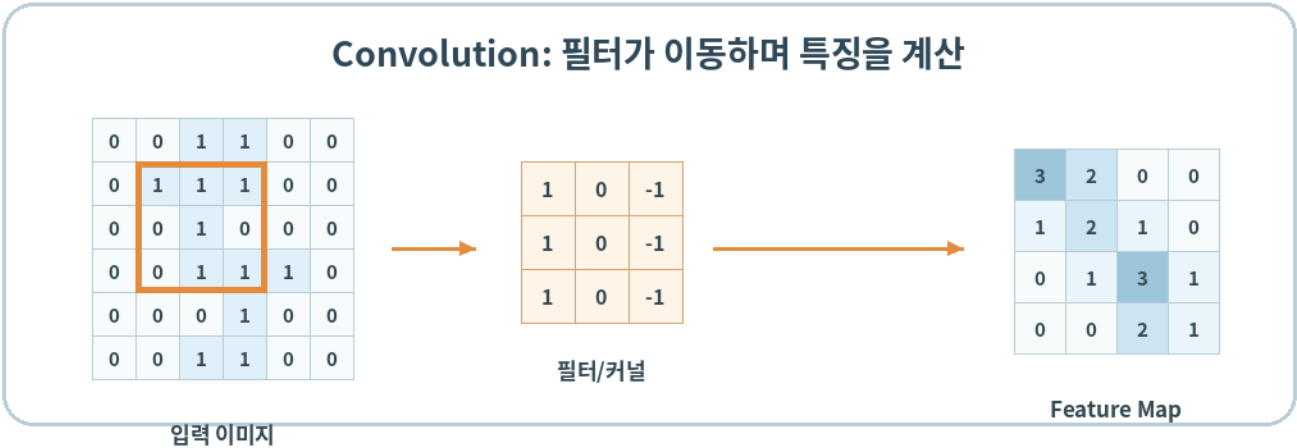
중요한 점은 CNN이 사람이 직접 정한 필터만 사용하는 것이 아니라, 데이터와 손실 함수를 통해 필요한 필터를 스스로 학습한다는 것이다.



5. Feature Map: 특징이 나타난 위치를 저장한다

Feature Map은 필터가 이미지의 각 위치에서 얼마나 강하게 반응했는지를 나타내는 출력이다. 값이 크다는 것은 해당 위치에 필터가 찾는 패턴이 강하게 존재한다는 뜻이다.

따라서 Feature Map은 단순한 숫자 배열이 아니라, 특정 특징의 위치 정보를 가진 지도라고 볼 수 있다.



6. Stride와 Padding

Stride는 필터가 한 번에 몇 칸씩 이동하는지를 정한다. Stride가 작으면 더 촘촘히 보므로 출력 Feature Map이 크고, Stride가 크면 빠르게 이동하므로 출력 크기가 작아진다.

Padding은 입력 이미지 가장자리에 0을 덧붙이는 방법이다. Padding을 사용하면 필터가 테두리 픽셀도 충분히 볼 수 있고, 출력 크기가 너무 빠르게 줄어드는 것을 막을 수 있다.

Stride와 Padding이 출력 크기를 바꾼다



$$\text{Output size} = \text{floor}((\text{Input} + 2P - K) / S) + 1$$

7. 출력 크기 계산 예시

CNN 구조를 설계할 때는 각 층을 지나며 이미지 크기가 어떻게 변하는지 확인해야 한다. 크기 계산을 하지 않으면 마지막 Flatten 또는 Fully Connected Layer에서 차원이 맞지 않는 문제가 생긴다.

입력 크기가 32, 필터 크기가 3, padding이 1, stride가 1이면 출력 크기는 다시 32가 된다.

주의

Padding과 stride는 단순 옵션이 아니라 정보량, 계산량, 출력 차원에 직접 영향을 주는 설계 요소이다.

$$\text{floor}((32 + 2 \times 1 - 3) / 1) + 1 = 32$$

입력	필터 K	Padding P	Stride S	출력	해석
32	3	0	1	30	크기 감소
32	3	1	1	32	크기 유지
32	3	1	2	16	절반 수준 감소

8. CNN에서도 Activation Function이 필요하다

Convolution만 반복하면 여전히 선형 계산의 조합이 된다. 복잡한 이미지 패턴을 표현하려면 Convolution 뒤에 ReLU 같은 비선형 활성화 함수가 필요하다.

ReLU는 음수 값을 0으로 만들고 양수 값은 그대로 통과시킨다. CNN에서는 Conv → ReLU 순서가 자주 사용된다.

해석

Conv가 “무엇이 보이는가?”를 계산한다면, ReLU는 그 반응을 비선형적으로 정리해 다음 층이 더 복잡한 조합을 만들 수 있게 한다.

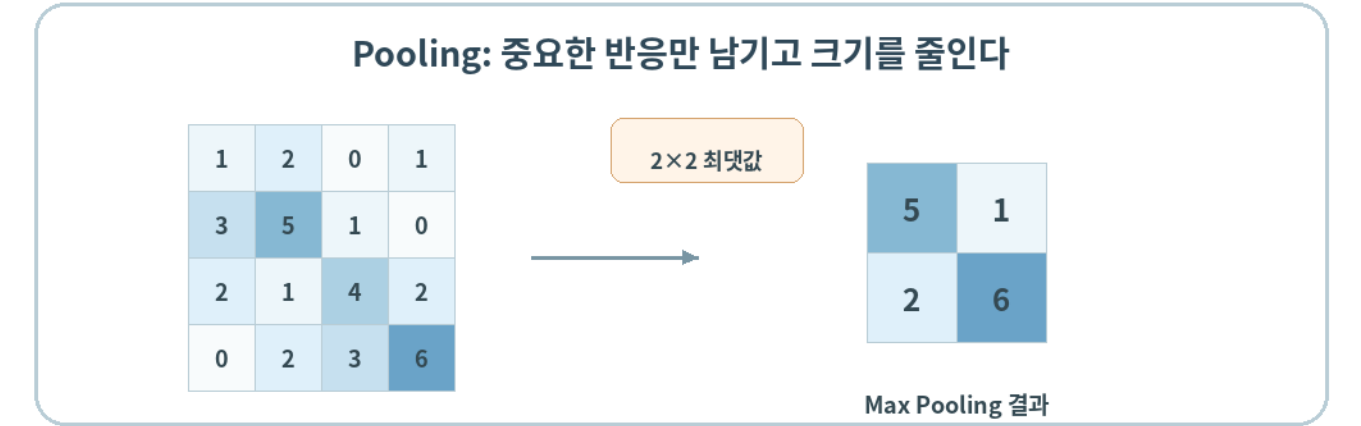
$$\text{ReLU}(x) = \max(0, x)$$

구성	역할	직관
Convolution	지역 특징을 계산	패턴 탐지
ReLU	비선형성 추가	필요한 반응만 통과
Pooling	공간 크기 축소	중요한 반응 요약

9. Pooling: 크기를 줄이고 중요한 반응을 남긴다

Pooling은 Feature Map의 공간 크기를 줄이는 연산이다. 가장 많이 쓰이는 Max Pooling은 작은 영역에서 가장 큰 값을 선택한다.

Pooling을 사용하면 계산량이 줄고, 작은 위치 변화에 덜 민감한 표현을 만들 수 있다.

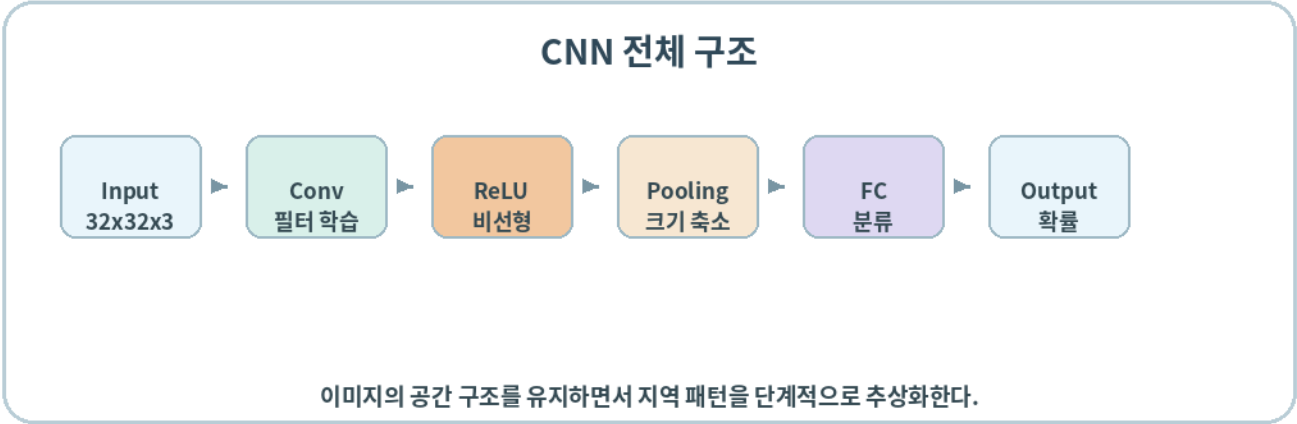


이런 Feature Map		
Pooling 종류	계산 방식	직관
Max Pooling	영역 내 최댓값 선택	가장 강한 특징 반응 보존
Average Pooling	영역 내 평균값 선택	전체적인 평균 반응 요약

10. CNN 기본 블록

CNN은 보통 Convolution, ReLU, Pooling을 하나의 기본 블록처럼 반복해서 사용한다. 앞쪽 층에서는 단순한 선이나 모서리를 찾고, 뒤쪽 층에서는 더 복잡한 형태와 의미 있는 패턴을 조합한다.

마지막에는 Feature Map을 펼친 뒤 Fully Connected Layer 또는 Global Average Pooling을 통해 클래스 점수를 만든다.



Input → Conv → ReLU → Pooling → FC → Output

구간	역할	결과
Feature Extractor	이미지에서 특징 추출	Feature Map
Classifier	추출된 특징으로 분류	클래스 점수 또는 확률

11. CNN Forward Propagation

CNN의 Forward Propagation은 입력 이미지가 Conv, ReLU, Pooling, 분류층을 순서대로 통과하여 예측값이 만들어지는 과정이다. 이 단계에서는 아직 가중치를 수정하지 않고 현재 필터와 가중치로 계산만 수행한다.

직관

Forward는 “현재 모델이 이미지를 어떻게 해석하는지 확인하는 계산 과정”이다. 학습은 이 결과와 정답을 비교한 뒤 시작된다.

$x \rightarrow \text{Feature Maps} \rightarrow \text{Class Scores} \rightarrow \text{Softmax Probabilities}$

단계	출력	의미
Input	$H \times W \times C$	원본 이미지 텐서
Conv + ReLU	Feature Maps	지역 패턴 반응
Pooling	작아진 Feature Maps	중요 정보 요약
Classifier	Class Scores	각 클래스에 대한 점수
Softmax	Probabilities	분류 확률

12. CNN 분류에서의 Loss Function

이미지 분류 CNN은 보통 Cross Entropy Loss를 사용한다. 모델은 각 클래스에 확률을 주고, Cross Entropy는 정답 클래스에 높은 확률을 주지 못할수록 손실을 크게 만든다.

따라서 학습은 정답 클래스의 확률을 높이는 방향으로 진행된다.



$$L = - \sum_k y_k \log(\hat{y}_k)$$

정답 클래스 확률	손실	해석
0.95	작음	정답에 확신이 높음
0.50	중간	헷갈리는 상태
0.05	큼	정답을 거의 선택하지 않음

13. CNN에서 Backpropagation은 무엇을 고치는가?

Backpropagation은 손실을 줄이기 위해 각 필터와 분류층 가중치가 오차에 얼마나 영향을 주었는지 계산한다. DNN에서는 일반 가중치를 고쳤다면, CNN에서는 Convolution 필터의 값도 함께 고친다.

어떤 필터가 잘못된 특징에 반응했다면 그 필터의 값이 수정된다. 반대로 정답을 맞히는 데 도움이 된 필터는 더 유용한 방향으로 강화된다.

중요한 해석

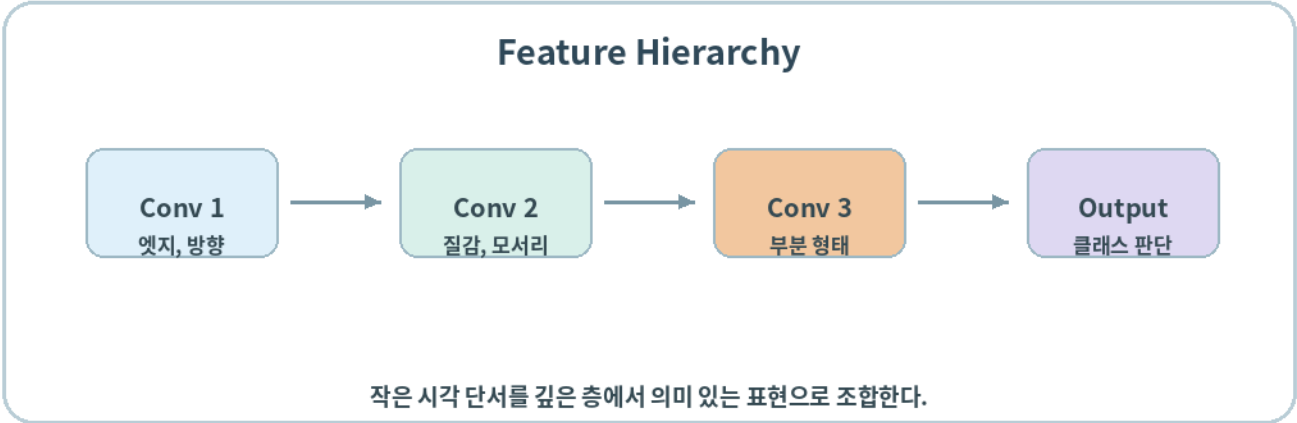
CNN 학습은 필터를 사람이 직접 설계하는 과정이 아니라, 손실을 줄이는 방향으로 필터가 자동 조정되는 과정이다.

$$\text{Filter}_{\text{new}} = \text{Filter}_{\text{old}} - \eta \cdot \partial L / \partial \text{Filter}$$

대상	수정되는 것	의미
Convolution Layer	필터 값	어떤 패턴을 볼지 조정
Fully Connected Layer	분류 가중치	특징을 어떤 클래스로 연결할지 조정
Bias	기준점	반응 기준 이동

14. CNN이 학습하는 표현의 계층 구조

CNN은 모든 층에서 같은 수준의 특징을 찾지 않는다. 앞쪽 층은 선, 방향, 색 대비 같은 단순한 특징에 반응하고, 뒤쪽 층은 질감, 형태, 물체의 부분처럼 더 추상적인 특징에 반응한다.



층 위치	주로 학습하는 특징	예시
얕은 층	저수준 특징	선, 모서리, 색 대비
중간 층	부분 패턴	질감, 작은 모양
깊은 층	고수준 특징	객체 부품, 클래스 단서

15. 예시: MNIST와 CIFAR-10

MNIST는 28×28 흑백 손글씨 숫자 이미지이고, CIFAR-10은 32×32 컬러 이미지이다. MNIST는 구조가 단순해서 DNN도 어느 정도 잘 동작하지만, CIFAR-10은 색상, 배경, 객체 형태가 다양하므로 CNN의 장점이 더 중요해진다.

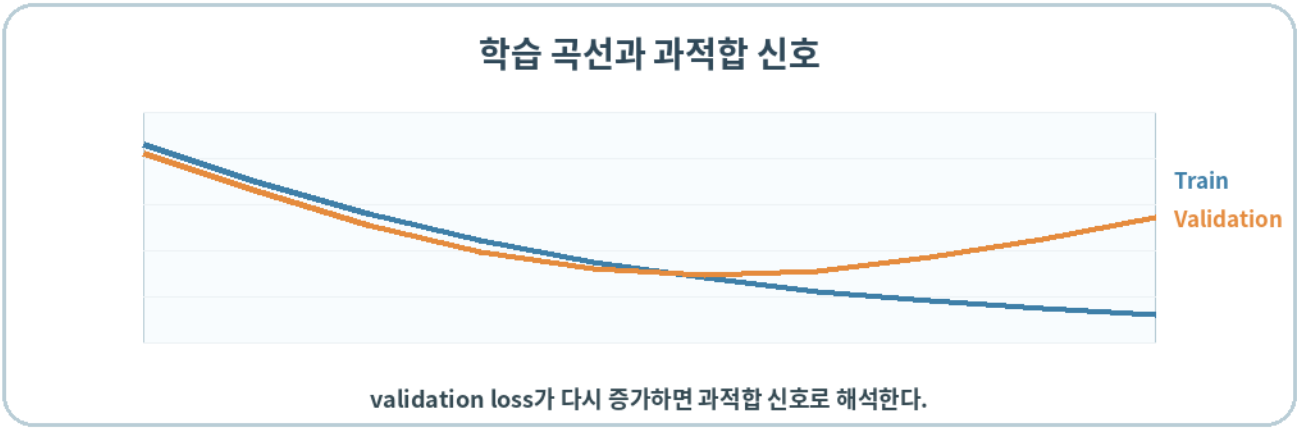


데이터셋	입력 형태	클래스	CNN 관점
MNIST	$28 \times 28 \times 1$	숫자 0-9	획, 굴곡, 모서리 패턴이 중요
Fashion-MNIST	$28 \times 28 \times 1$	의류 10종	윤곽과 형태 차이가 중요
CIFAR-10	$32 \times 32 \times 3$	동물/탈것 10종	색, 질감, 부분 형태가 중요

16. CNN의 과적합 문제

CNN은 이미지 특징을 잘 학습하지만, 데이터가 적거나 모델이 너무 크면 학습 데이터를 외워버릴 수 있다. 특히 훈련 정확도는 매우 높는데 검증 정확도가 낮다면 과적합을 의심해야 한다.

이미지에서는 같은 객체라도 위치, 각도, 밝기, 배경이 달라질 수 있다. 학습 데이터가 이런 다양성을 충분히 포함하지 못하면 실제 환경에서 성능이 떨어진다.



상태	Train 성능	Validation 성능	해석
과소적합	낮음	낮음	패턴을 충분히 배우지 못함
정상 학습	높음	높음	일반화가 잘 됨
과적합	매우 높음	낮음	학습 데이터를 외움

17. CNN에서 과적합을 줄이는 방법

과적합을 줄이려면 모델이 특정 훈련 이미지의 세부 노이즈를 외우지 않고, 새로운 이미지에도 통하는 시각 규칙을 배우게 만들어야 한다. 이를 위해 데이터 증강, Dropout, Weight Decay, Early Stopping을 함께 사용한다.

Data Augmentation: 같은 정답을 유지한 채 다양화



원본



회전



이동



밝기



반전

위치, 각도, 밝기 변화에 강한 모델을 만드는 데 도움을 준다.

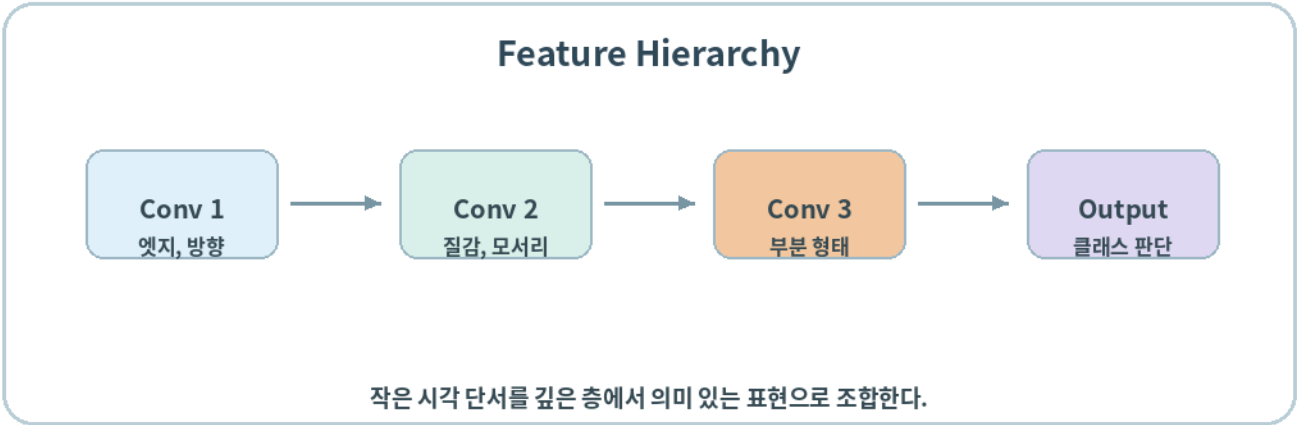
방법	핵심 아이디어	CNN에서의 효과
Data Augmentation	이미지를 변형해 다양성 증가	위치, 각도, 밝기 변화에 강해짐
Dropout	일부 뉴런을 임시로 끄	분류층이 특정 특징에만 의존하지 않음
Weight Decay	가중치가 지나치게 커지는 것 억제	복잡한 결정 규칙 완화
Early Stopping	검증 성능 악화 전 중단	외우기 전에 학습 종료

18. 대표 CNN 구조를 보는 방법

LeNet, VGG, ResNet처럼 유명한 CNN 구조는 세부 설계가 다르지만 기본 관점은 비슷하다. 이미지를 여러 Convolution 블록으로 처리해 Feature Map을 만들고, 마지막에 분류기를 통해 클래스를 예측한다.

입문자 관점

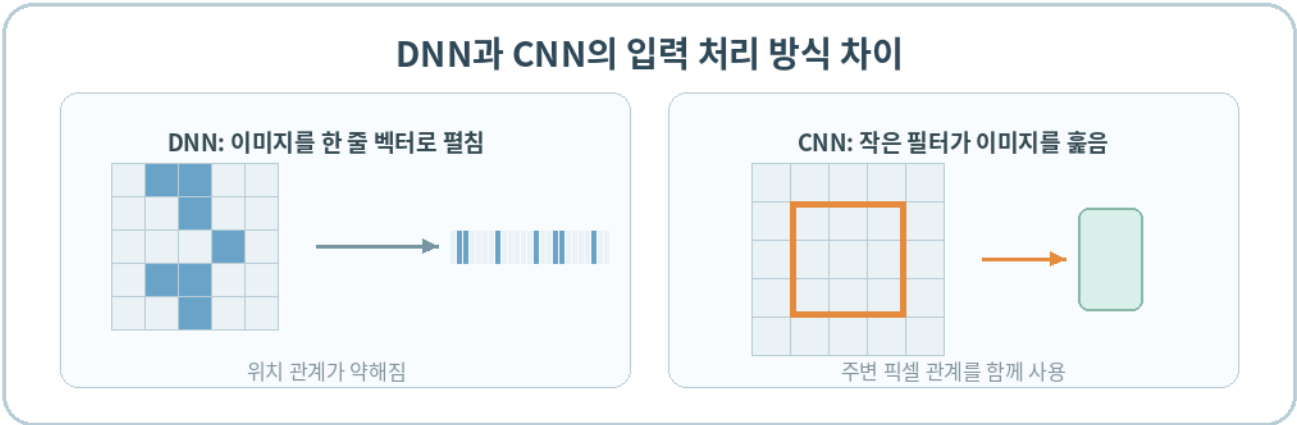
처음부터 복잡한 모델 이름을 외울 필요는 없다. Conv 블록이 어떻게 특징을 만들고, 깊어질수록 특징이 어떻게 추상화되는지를 먼저 이해하는 것이 중요하다.



구조	핵심 특징	교육용 해석
LeNet	초기 CNN 구조, Conv와 Pooling 반복	MNIST 예제에 적합
VGG	3×3 Conv를 깊게 쌓음	작은 필터를 반복하면 깊은 표현 가능
ResNet	Skip Connection 사용	깊은 네트워크의 학습 안정화

19. DNN vs CNN 비교

DNN과 CNN은 모두 가중치를 학습하고, forward, loss, backpropagation, update 과정을 거친다. 차이는 입력을 바라보는 방식에 있다. DNN은 일반 벡터 입력에 적합하고, CNN은 이미지처럼 공간 구조가 중요한 데이터에 적합하다.



20. CNN 학습 결과를 볼 때 체크리스트

CNN 성능이 낮다고 바로 층을 깊게 만들기보다, 입력 형태, 출력 차원, 손실 함수, 학습 곡선, 과적합 여부를 순서대로 확인해야 한다. 특히 이미지 데이터에서는 전처리와 데이터 증강 설정이 성능에 큰 영향을 준다.

플랫폼 연결

블록 코딩 플랫폼에서는 사용자가 Conv 필터 수, kernel size, stride, padding, pooling, optimizer를 직접 바꾸게 된다. 따라서 각 설정이 출력 크기와 학습 성능에 어떤 영향을 주는지 시각적으로 보여주는 것이 중요하다.

점검 항목	확인 질문
입력 형태	이미지가 $H \times W \times C$ 형태로 올바르게 들어가는가?
정규화	픽셀값 범위가 0-1 또는 표준화된 범위인가?
출력 뉴런 수	클래스 수와 출력 차원이 일치하는가?
Loss Function	다중 분류라면 Cross Entropy를 쓰고 있는가?
학습 곡선	train loss와 validation loss를 함께 보는가?
과적합	train 성능만 높고 validation 성능은 낮지 않은가?
시각화	Feature Map 또는 예측 확률을 확인했는가?

21. 전체 요약

한 줄 정리

CNN은 이미지를 작은 지역 패턴으로 나누어 보고, 그 패턴들을 깊은 층에서 조합하여 정답을 예측하는 이미지 특화 신경망이다.

- CNN은 이미지의 공간 구조를 유지하면서 지역 특징을 추출하는 신경망이다.
- Convolution은 필터가 이미지를 이동하며 작은 영역의 패턴을 계산하는 과정이다.
- Filter는 학습되는 가중치이며, 특정 시각 패턴에 반응하는 탐지기 역할을 한다.
- Feature Map은 필터가 각 위치에서 얼마나 강하게 반응했는지를 저장한 지도이다.
- Stride는 필터의 이동 간격이고, Padding은 가장자리 정보를 보존하기 위해 0을 추가하는 방법이다.
- Pooling은 중요한 반응을 남기면서 Feature Map의 크기를 줄인다.
- CNN은 얇은 층에서 선과 모서리를, 깊은 층에서 더 추상적인 객체 단서를 학습한다.
- 이미지 분류에서는 Cross Entropy Loss를 사용해 정답 클래스 확률을 높이는 방향으로 학습한다.
- 과적합을 막기 위해 Data Augmentation, Dropout, Weight Decay, Early Stopping을 사용할 수 있다.